FULL STACK WEB DEVELOPMENT

PROJECT REPORT



QuantLens

A Mathematical Stock Analysis Platform

Department	CSE (Artificial Intelligence & Machine Learning)
Subject	Full Stack Web Development
Subject Code	22CAC11N
Project Title / Case Study	QuantLens: A Mathematical Stock Analysis Platform

Submitted by

S. No.	Student Name	Roll Number
1	Mohammed Raahil Ali	160124748052
2	Shaik Mohammed Younus Saqib	160124748059

Under the guidance of

Ms. Falak Naaz

Faculty, Department of CSE (AI & ML)

Date: 29 October 2026

1. Cover Page

The cover page above lists the department, subject, subject code, project title, the two team members with their roll numbers, the guiding faculty member and the date of submission, as required by the report format.

Table 1: Project at a glance

Item	Details
Project	QuantLens: A Mathematical Stock Analysis Platform
Team members	Mohammed Raahil Ali (160124748052), Shaik Mohammed Younus Saqib (160124748059)
Faculty guide	Ms. Falak Naaz
Type of application	Full-stack single-page web application with REST API, database, cache, job queue and payment gateway
Source code	github.com/mra-2007/ventureiq (branch quantlens)

2. Abstract (Short Overview)

QuantLens is a full-stack web application in which a user searches for any listed stock and immediately sees the classic quantitative indicators and risk metrics computed from its historical prices, drawn on interactive charts. The case study selected is a mathematical stock analysis platform: the application uses no artificial intelligence and gives no opinions; every number is produced by a published formula from technical analysis, risk management or portfolio theory, and the formula is shown next to the number.

The frontend is a React 19 single-page application built with Vite and React Router, using TradingView Lightweight Charts for candlestick charts and Recharts for analytical charts. The backend is a Node.js and Express 5 REST API with a pure-JavaScript math engine. Price data comes from Yahoo Finance (Alpha Vantage is supported as an alternative). PostgreSQL stores users, watchlists, saved strategies and price alerts; Redis caches market data and computed results; BullMQ runs Monte Carlo simulations, backtests and portfolio optimisation as background jobs; JSON Web Tokens secure the API; and Razorpay Checkout handles payment for the Pro plan.

The finished system computes SMA, EMA, MACD, Bollinger Bands, RSI and the Stochastic oscillator; volatility, Sharpe and Sortino ratios, maximum drawdown, Value at Risk and beta; a correlation heatmap for 2 to 5 stocks; a Markowitz efficient frontier; a linear-regression trendline; a 1,000-path Monte Carlo fan chart; and a rule-based strategy backtester compared with buy-and-hold. All 15 automated tests of the math engine pass, and the complete stack was verified end to end with PostgreSQL, Redis and BullMQ.

3. Problem Statement

Retail investors and students who want to understand a stock usually see only its price and a simple line chart. Quantitative measures such as volatility, drawdown, Value at Risk, correlation with other stocks, or the historical performance of a trading rule require spreadsheets, programming knowledge or expensive professional terminals. Free websites that show indicators rarely explain how a number was produced, and many now present AI-generated opinions instead of verifiable mathematics. Free market-data APIs also impose strict rate limits, so a naive application that refetches data for every chart quickly stops working.

Problem statement: *"To design and develop a web application that fetches historical price data for any stock, computes standard quantitative indicators, risk metrics, portfolio statistics, forecasts and strategy backtests using transparent mathematical formulas, presents them on interactive charts, and stores each user's watchlists, saved strategies and price alerts securely, while staying within the rate limits of free market-data APIs."*

Table 2: How QuantLens addresses the gap

Need	Typical free tools	QuantLens
Indicators on a chart	Available, formulas hidden	Available, each with its formula
Risk metrics (VaR, drawdown, beta)	Rare or paywalled	Computed for every stock
Compare several stocks	Side-by-side prices only	Correlation heatmap, beta and risk table
Test a trading rule	Requires coding	Point-and-click rule builder
Portfolio optimisation	Professional software	Live Markowitz efficient frontier
Forecast uncertainty	Single target price	1,000-path Monte Carlo fan chart

4. Objectives

Based on the project scope, the objectives of QuantLens are:

1. To design responsive web pages using HTML and CSS with a consistent design system (colour tokens, typography and a layout grid) that works from mobile phones to desktops.
2. To develop client-side scripting in JavaScript and React for stock search with autocomplete, chart interaction, form validation and polling of background jobs.
3. To design and develop a single-page application with client-side routing (React Router), lazy-loaded pages and a shared authentication context.
4. To implement backend services as a REST API in Node.js and Express with JWT authentication, input validation, rate limiting and a Free/Pro plan model.
5. To design the PostgreSQL database schema for users, watchlists, saved strategies and price alerts, and to use Redis to cache price data and computed results.

6. To implement a mathematical engine that computes every indicator in a single O(n) pass using rolling windows, and to run heavy computations (Monte Carlo, backtests, efficient frontier) in a background job queue.
7. To integrate the Razorpay payment gateway for the Pro subscription with server-side signature verification.
8. To verify correctness with automated tests that compare the optimised algorithms against naive reference implementations.

5. System Design

5.1 System architecture

QuantLens follows a three-tier client–server architecture. The React single-page application in the browser communicates only with the Express REST API over HTTPS using JSON, sending the user's JWT as a Bearer token. The API answers inexpensive requests (search, analysis, comparison) directly and places expensive ones on a BullMQ queue backed by Redis. A worker process takes jobs from the queue, runs them through the same quant engine and stores the result in Redis. PostgreSQL holds all permanent user data, Yahoo Finance supplies prices on a cache miss, Razorpay processes Pro payments, and an alert checker emails users through SMTP.

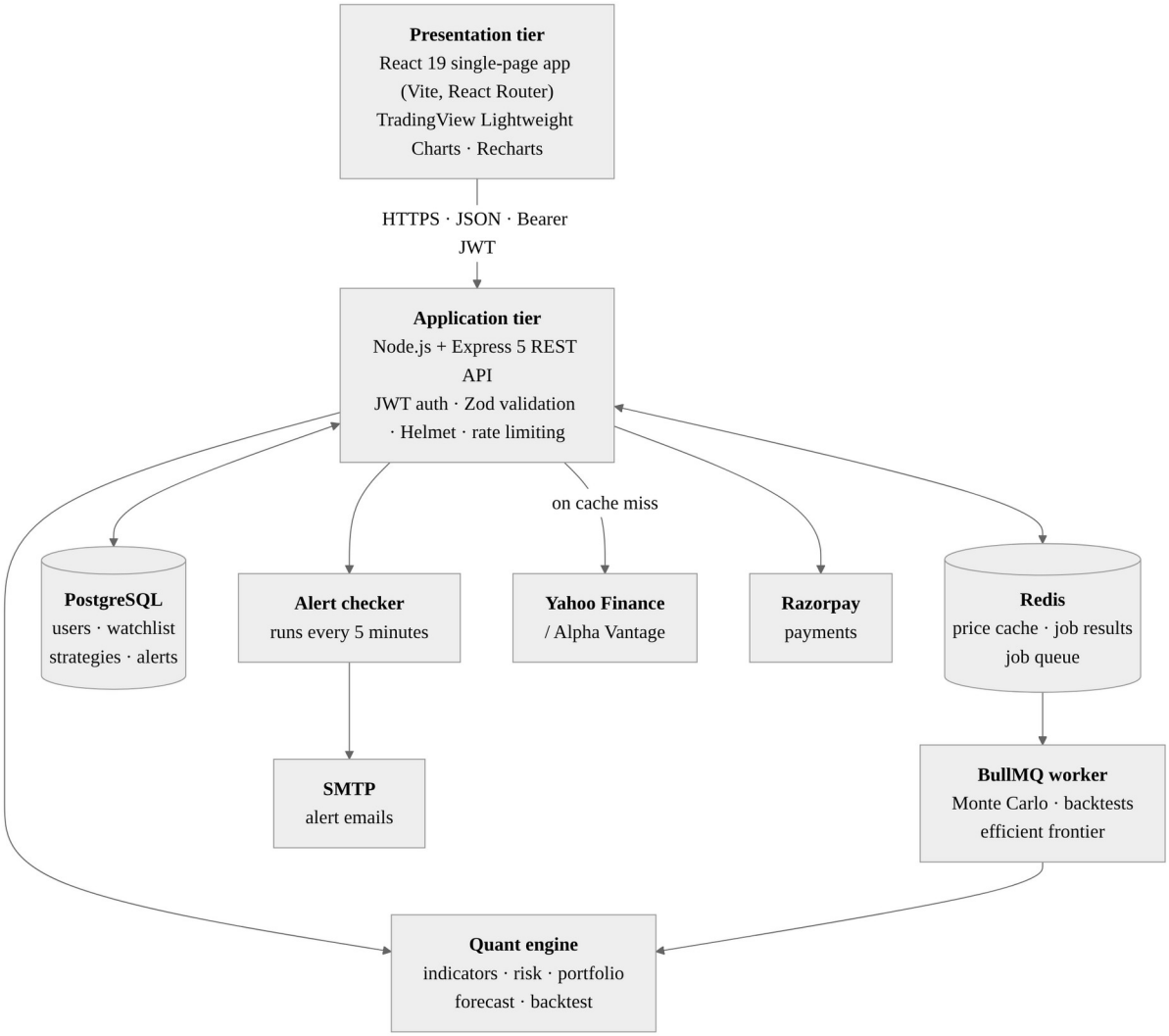


Figure 1: QuantLens system architecture

5.2 Technology stack

Table 3: Technology stack

Layer	Technology and purpose
Frontend	React 19, Vite 8, React Router 7; hand-written CSS design system (no UI framework)
Charts	TradingView Lightweight Charts 5 for candlesticks, overlays and oscillator panes; Recharts 3 for the fan chart, frontier, equity curve and histograms
Backend	Node.js 22, Express 5, Zod (validation), Helmet (security headers), express-rate-limit, CORS
Math engine	Pure JavaScript modules: indicators, stats, risk, portfolio, forecast, backtest, analyze
Market data	yahoo-finance2 (default, no API key); Alpha Vantage (optional); seeded demo data for offline use
Database	PostgreSQL through the pg driver, with an in-memory fallback for local development
Cache and queue	Redis (ioredis) for price and result caching; BullMQ for background jobs

Layer	Technology and purpose
Authentication	JSON Web Tokens (jsonwebtoken) and bcrypt password hashing (bcryptjs)
Payments	Razorpay Checkout with HMAC-SHA256 signature verification on the server
Email	Nodemailer (SMTP) for price-alert emails
Testing and tools	node:test, ESLint, Playwright (UI screenshots), Git

5.3 Functional modules

Table 4: Functional modules

Module	Responsibility
Authentication	Register, sign in, JWT issue and verification, plan (Free/Pro) on every request
Market data	Symbol search, daily OHLCV history for 6 months to 5 years, Redis caching, provider switch
Analysis	Indicators, oscillators, risk metrics, regression trend and plain-language readings for one stock
Comparison	Aligns 2–5 price series on common dates; correlation matrix, beta, growth and risk table
Forecasting	Monte Carlo simulation of 1,000 price paths with percentile bands and ending-price histogram
Backtesting	Rule builder, next-bar execution, stop loss, take profit, fees, equity curve and trade list
Portfolio	Annualised moments, 4,000 sampled portfolios, efficient frontier, max-Sharpe and min-volatility weights
Watchlist and alerts	Per-user symbols with sparklines; email when a price target is crossed
Billing	Razorpay order creation, checkout and signature verification to activate Pro

5.4 Database design

The relational schema has four tables. Every child table references users(id) with ON DELETE CASCADE so deleting an account removes all its data. Strategy rule sets and their last result are stored as JSONB, which keeps the schema stable while the rule builder evolves. A partial index on active alerts keeps the five-minute alert check fast.

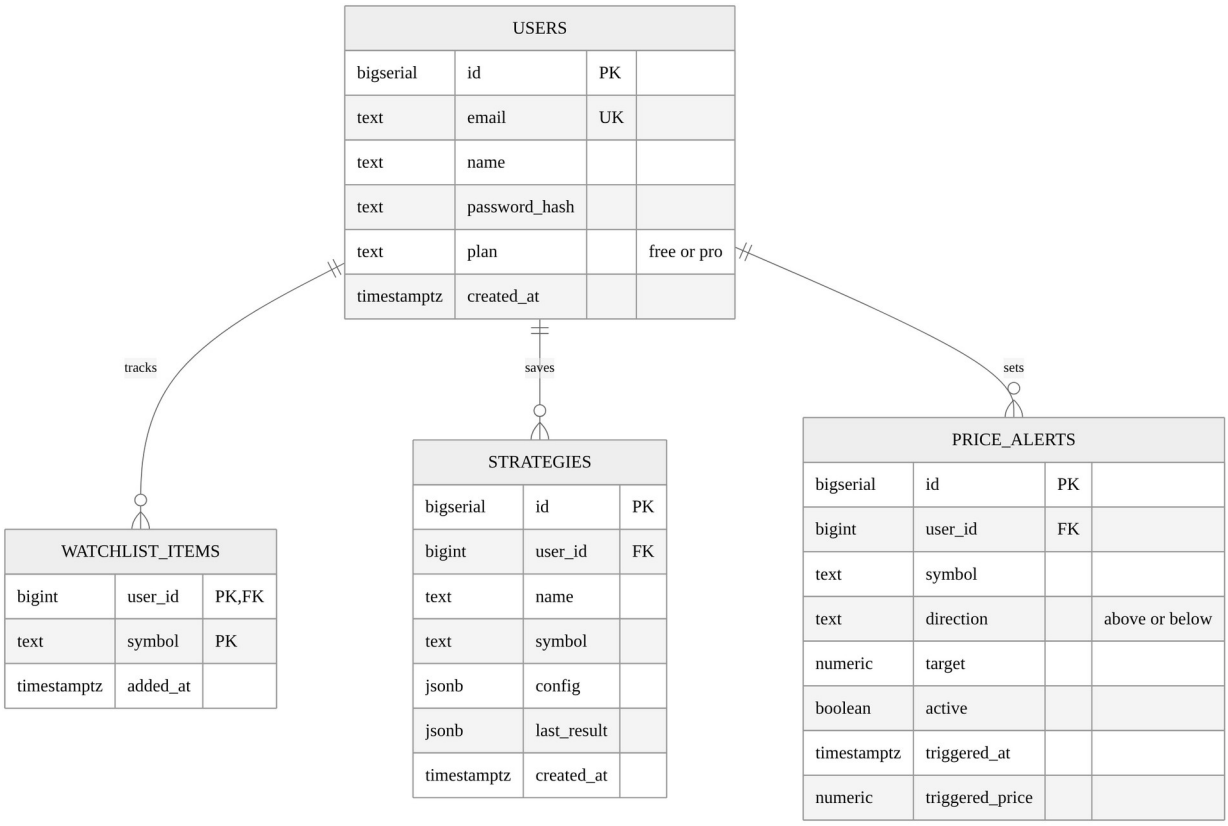


Figure 2: Entity–relationship diagram of the PostgreSQL schema

Table 5: Database tables

Table	Key columns	Constraints and notes
users	id, email, name, password_hash, plan, created_at	email unique; plan is 'free' or 'pro'
watchlist_items	user_id, symbol, added_at	Primary key (user_id, symbol); Free plan limited to 5
strategies	id, user_id, name, symbol, config, last_result	JSONB rule set; Pro only
price_alerts	id, user_id, symbol, direction, target, active, triggered_at	direction is 'above' or 'below'; Pro only

5.5 REST API design

Table 6: REST API endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register, /api/auth/login	Public	Create account or sign in; returns a JWT
GET	/api/auth/me	User	Current user and plan
GET	/api/search?q=	Public	Symbol autocomplete
GET	/api/stocks/:symbol/analysis	Public	Candles, indicators, risk metrics and trend

Method	Endpoint	Access	Purpose
GET	/api/compare?symbols=	Public	Correlation, beta and performance for 2–5 stocks
POST	/api/jobs/montecarlo	Public	Queue a 1,000-path simulation
POST	/api/jobs/backtest	Pro	Queue a strategy backtest
POST	/api/jobs/frontier	Pro	Queue efficient-frontier optimisation
GET	/api/jobs/:id	Public	Poll job state and result
GET, POST, DELETE	/api/watchlist	User	Manage the watchlist
GET, POST, DELETE	/api/strategies, /api/alerts	Pro	Saved strategies and price alerts
GET	/api/billing/config	Public	Payment provider, price and test-mode flag
POST	/api/billing/order, /api/billing/verify	User	Create a Razorpay order; verify payment and grant Pro

5.6 Background job flow

Long computations never block the web request. The API checks whether an identical job has already been computed (results are keyed by a hash of the inputs); if not, it queues the job and returns a job ID at once with HTTP 202. The browser polls the job endpoint with increasing delays until the result is ready.

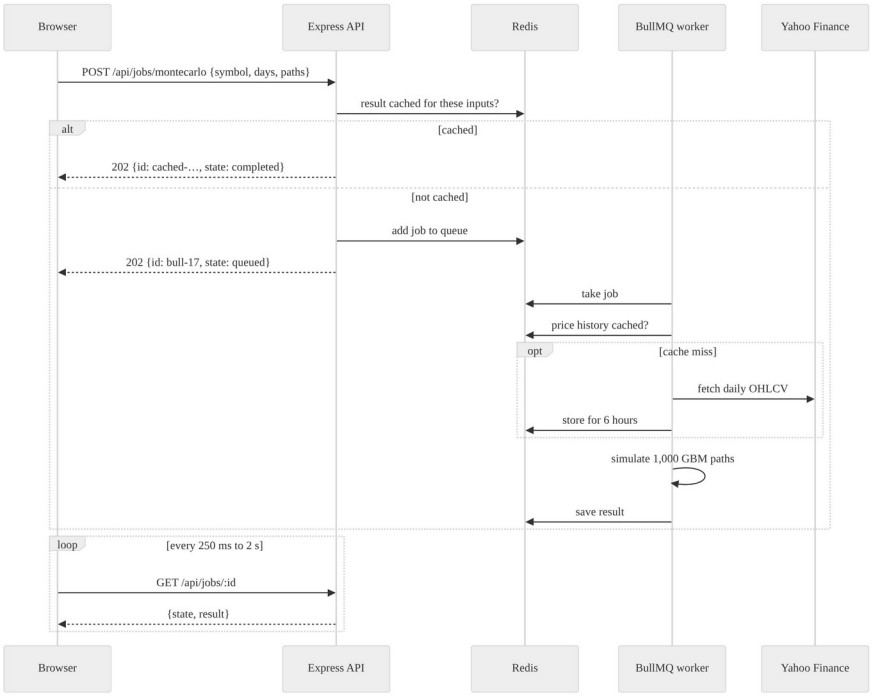


Figure 3: Sequence diagram of a Monte Carlo job through Redis and BullMQ

5.7 Payment flow

The Pro plan is purchased through Razorpay. The server creates the order using its secret key, the browser opens Razorpay Checkout, and after payment the server recomputes the HMAC-SHA256 signature of the order and payment IDs. Only if the signature matches does it upgrade the account and issue a new token, so a user cannot unlock Pro by faking a successful payment in the browser.

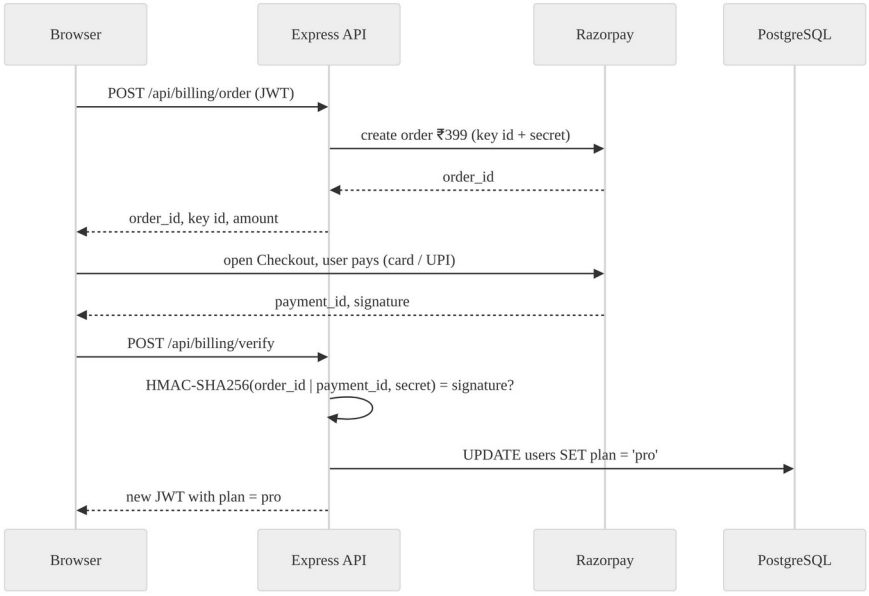


Figure 4: Sequence diagram of the Razorpay payment and verification flow

5.8 User interface design

The interface uses a deliberate dark “ink-blue” theme rather than plain black, with chalk-white text, sodium-amber for anything forecast or computed, and teal and coral for gains and losses. Headings and numbers use Hanken Grotesk with tabular figures so columns of numbers align; formulas are typeset in STIX Two italic so every metric can display the mathematics behind it. The layout is a 1,240-pixel grid that collapses to a single column on phones.

Table 7: Design tokens

Token	Value	Used for
Ink	#0E1726	Page background
Panel	#131E31	Cards and chart panels
Chalk	#E6ECF5	Primary text and current price line
Amber	#F2B544	Forecasts, medians, primary buttons, the efficient frontier
Teal / Coral	#3DD6B5 / #FF6B6B	Up and down moves, gains and losses

Table 8: Frontend routes

Route	Page
/	Landing page with animated Monte Carlo hero, module overview and pricing

Route	Page
/login, /signup	Authentication forms
/s/:symbol	Stock analysis: chart, overlays, oscillators, risk, drawdown, readings, Monte Carlo, alerts
/compare	Compare 2–5 stocks: correlation heatmap, growth chart and risk table
/backtest	Strategy rule builder, equity curve, statistics and trade list (Pro)
/portfolio	Portfolio builder with live efficient frontier and optimal weights (Pro)
/dashboard	Watchlist with sparklines, saved strategies, price alerts and plan

6. Implementation

6.1 Project structure

```
QuantLens/
├── src/                      React single-page application
│   ├── pages/                Landing, Stock, Compare, Backtest, Portfolio, Dashboard,
Auth
│   ├── components/           PriceChart, FanChart, Heatmap, shared UI
│   └── lib/                  API client, auth context (JWT + Razorpay), formatting
├── server/                   Express REST API
│   ├── src/quant/            indicators, stats, risk, portfolio, forecast, backtest,
analyze
│   ├── src/lib/              market data, cache, auth middleware, alerts, demo data
│   ├── src/jobs/             BullMQ queue, worker, job handlers
│   ├── src/db/               schema.sql, migrate, PostgreSQL / in-memory repository
│   ├── src/routes/api.js     REST endpoints and billing
│   └── test/                 15 automated tests for the math engine
└── docker-compose.yml        local PostgreSQL and Redis
```

Listing 1: Folder structure

6.2 Indicator engine in O(n)

Every indicator is computed in a single pass over the price series. Moving averages keep a running sum, Bollinger Bands keep a running sum and sum of squares, EMA and RSI use recursive smoothing, and the Stochastic oscillator uses a monotonic deque to find the rolling highest high and lowest low in amortised constant time. This replaces the naive O(n·k) approach of recomputing each window.

```
export function sma(values, period) {
  const out = new Array(values.length).fill(null);
  let sum = 0;
  for (let i = 0; i < values.length; i++) {
    sum += values[i];
    if (i >= period) sum -= values[i - period];   // slide the window in O(1)
    if (i >= period - 1) out[i] = sum / period;
  }
  return out;
}
```

Listing 2: Simple moving average with a rolling sum

Table 9: Formulas implemented in the math engine

Metric	Formula used
EMA	$EMA_t = \alpha \cdot P_t + (1 - \alpha) \cdot EMA_{t-1}$, $\alpha = 2/(n + 1)$
MACD	$EMA_{12} - EMA_{26}$; signal = EMA_9 of MACD; histogram = MACD − signal
Bollinger Bands	$SMA_{20} \pm 2\sigma_{20}$
RSI	100 − 100/(1 + RS), RS = Wilder-smoothed average gain / average loss
Stochastic %K	$100 \cdot (C - L_{14})/(H_{14} - L_{14})$; %D = 3-day SMA of %K
Volatility	σ(daily returns)·√252
Sharpe ratio	(252·μ − r_f) / (σ·√252)
Sortino ratio	(252·μ − r_f) / (σ_downside·√252)
Maximum drawdown	min over t of (P_t / running peak − 1)
Value at Risk (95%)	Historical: −5th percentile of returns; parametric: −(μ − 1.645σ)
Expected shortfall	−mean of returns at or below the 5th percentile
Beta	cov(r, r_index) / var(r_index)
Regression trend	Least squares $\hat{y} = a + b \cdot t$, with R^2 and a ±2σ residual channel

6.3 Monte Carlo simulation

Future prices are simulated with geometric Brownian motion calibrated on two years of daily log returns. The engine runs 1,000 paths with a seeded random-number generator (mulberry32 with a Box–Muller transform), so results are reproducible. Each time step is stored in a Float64Array, and the engine returns the 5th, 25th, 50th, 75th and 95th percentiles per day for the fan chart, a histogram of ending prices and the probability of finishing above today's price.

```
for (let p = 0; p < paths; p++) {
  let logS = Math.log(s0);
  for (let t = 0; t < days; t++) {
    logS += mu + sigma * nextNormal();      // ln S(t+1) = ln S(t) + μ + σZ
    columns[t][p] = Math.exp(logS);
  }
}
```

Listing 3: Core of the Monte Carlo simulation

6.4 Efficient frontier (Markowitz)

Annualised mean returns and the covariance matrix are computed from daily returns aligned on common trading dates. The optimiser samples 4,000 long-only weight vectors from a Dirichlet distribution, computes each portfolio's expected return $w^T\mu$, volatility $\sqrt{(w^T\Sigma w)}$ and Sharpe ratio, and extracts the upper envelope as the efficient frontier together with the maximum-Sharpe and minimum-volatility portfolios. The capital market line is drawn from the risk-free rate through the maximum-Sharpe point. The frontier recomputes automatically, with a short debounce, whenever a stock is added or removed.

6.5 Strategy backtester

A strategy is a JSON rule set: buy when all entry conditions are true and sell when any exit condition is true, with optional stop loss, take profit and trading fees. Each condition compares an indicator (price, SMA, EMA, RSI, MACD, Bollinger band or Stochastic) with another indicator or a number using crosses above, crosses below, greater than or less than. To avoid look-ahead bias, signals are evaluated on the close of day t and filled at the open of day t + 1. Each distinct indicator series is computed once and memoised. Results include the equity curve, every trade, total return, CAGR, Sharpe ratio, maximum drawdown, win rate and time in the market, all compared with buy-and-hold.

```
{ "entry": [{ "left":  { "ind": "sma", "period": 50 },
              "op":    "crossesAbove",
              "right": { "ind": "sma", "period": 200 } }],
  "exit":  [{ "left":  { "ind": "sma", "period": 50 },
              "op":    "crossesBelow",
              "right": { "ind": "sma", "period": 200 } }],
  "stopLoss": 0.08, "feeBps": 5 }
```

Listing 4: The golden-cross strategy as stored in PostgreSQL (JSONB)

6.6 Efficiency: caching and background jobs

- **Price cache.** Price history is cached in Redis under px:provider:symbol:range for six hours, so repeated analyses never hit the data provider’s rate limit.
- **Result cache.** Job results are cached by a SHA-1 hash of their inputs; an identical request returns instantly without recomputation.
- **Job queue.** Monte Carlo, backtests and frontier optimisation run on a BullMQ queue with a separate worker; the browser polls with exponential back-off.
- **Code splitting.** Chart-heavy pages are lazy-loaded with React.lazy, so the landing page downloads only what it needs.
- **Graceful fallback.** Without Redis or PostgreSQL configured, the server switches to an in-memory cache, store and queue, so the project runs on any laptop with npm run dev.

6.7 Security and the business model

- **Passwords and sessions.** Passwords are hashed with bcrypt; sessions use signed JWTs with a seven-day expiry sent as a Bearer token.
- **Input validation.** Every request body is validated with Zod schemas; Helmet sets security headers; login and registration are rate-limited against brute-force attempts.
- **Plan enforcement.** Middleware enforces the plan on the server. Free users get all indicators, risk metrics, Monte Carlo and comparison; Pro unlocks backtesting, the portfolio optimiser, saved strategies, price-alert emails and an unlimited watchlist.
- **Payment verification.** Pro is granted only after the server verifies Razorpay’s HMAC-SHA256 signature using a constant-time comparison.

```
const expected = crypto.createHmac("sha256", keySecret)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`).digest("hex");
if (!crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(razorpay_signature)))
  return res.status(400).json({ error: "Payment could not be verified" });
const user = await db.users.setPlan(req.user.id, "pro");
```

Listing 5: Server-side verification of a Razorpay payment

6.8 Frontend implementation

The React application keeps the signed-in user and the billing configuration in a single AuthContext. A small API client attaches the JWT to every request and turns server errors into readable messages. The PriceChart component creates a Lightweight Charts instance with a candlestick series, a volume histogram, the selected overlays in the main pane and the chosen oscillator (RSI, MACD or Stochastic) in a second pane. Background jobs are submitted with a runJob helper that polls until completion and can be cancelled with an AbortController when the user navigates away.

```
export async function runJob(type, payload, { onState, signal } = {}) {
  const { id } = await api(`/jobs/${type}`, { method: "POST", body: payload, signal });
  let delay = 250;
  for (;;) {
    const job = await api(`/jobs/${id}`, { signal });
    onState?.(job.state);
    if (job.state === "completed") return job.result;
    if (job.state === "failed") throw new ApiError(job.error, 500);
    await new Promise((r) => setTimeout(r, delay));
    delay = Math.min(delay * 1.5, 2000);
  }
}
```

Listing 6: Submitting and polling a background job from React

7. Outputs

The screenshots in this section were captured from the running application. For repeatable screenshots the server ran in demo-data mode, which generates realistic simulated prices offline; with DATA_PROVIDER=yahoo the same screens show live market data.

7.1 Landing page and authentication

The landing page opens with a Monte Carlo simulation that draws itself path by path from today’s price, next to the search box. Below it, each of the six engines lists the metrics it computes with their formulas, followed by the Free and Pro plans.

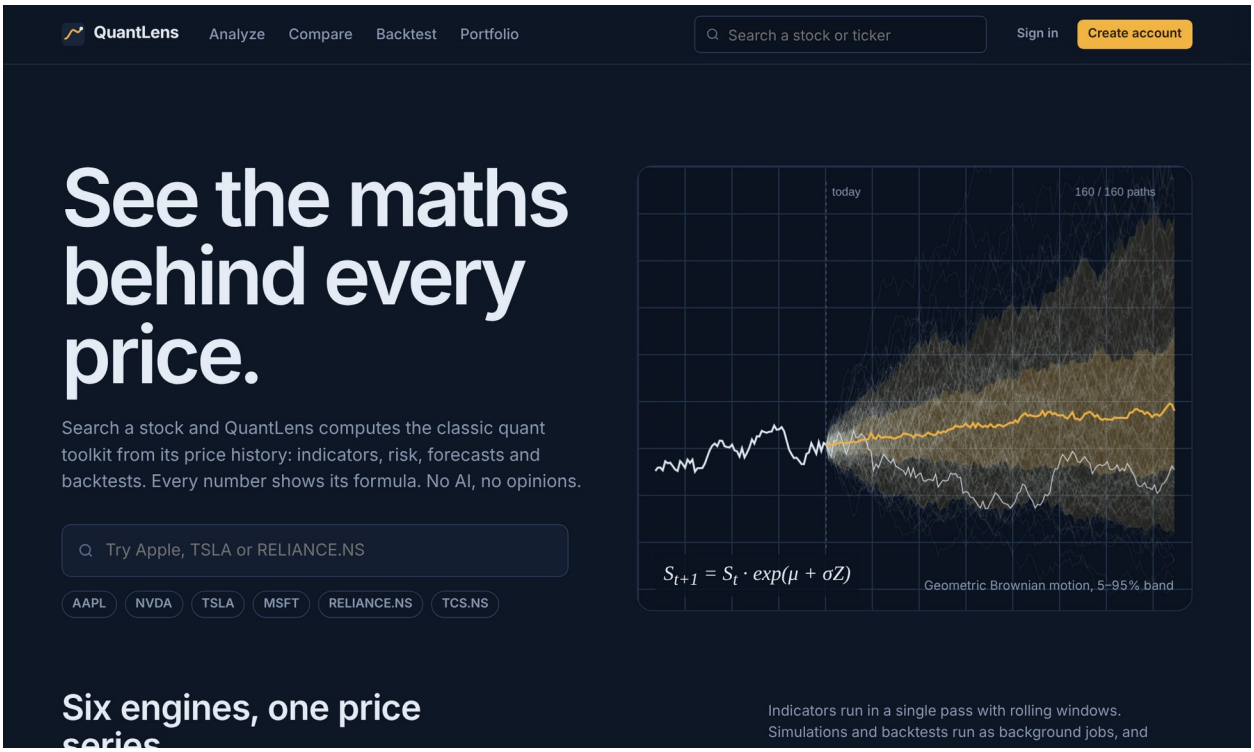


Figure 5: Landing page with animated Monte Carlo hero and stock search

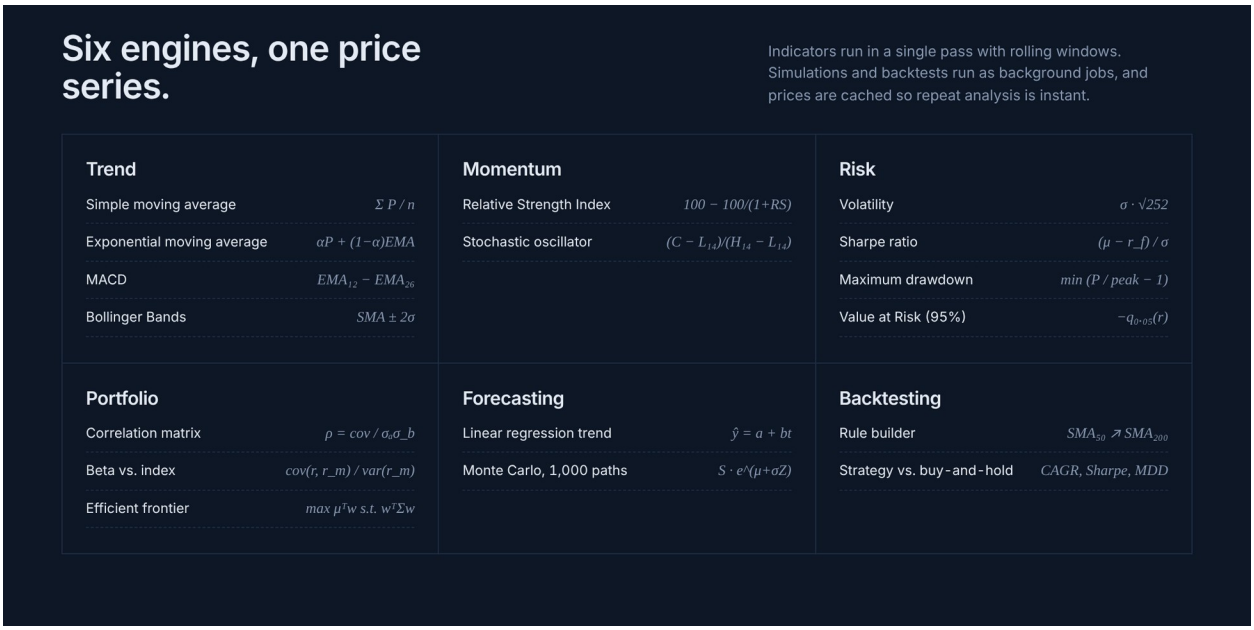


Figure 6: The six computation engines, each metric shown with its formula

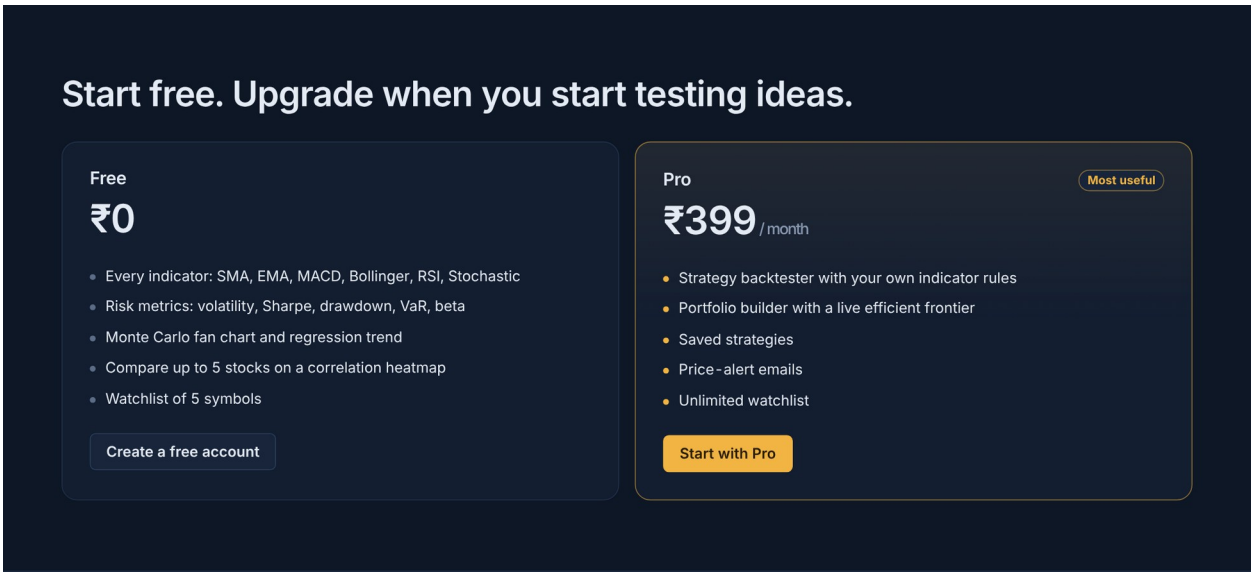


Figure 7: Free and Pro plans

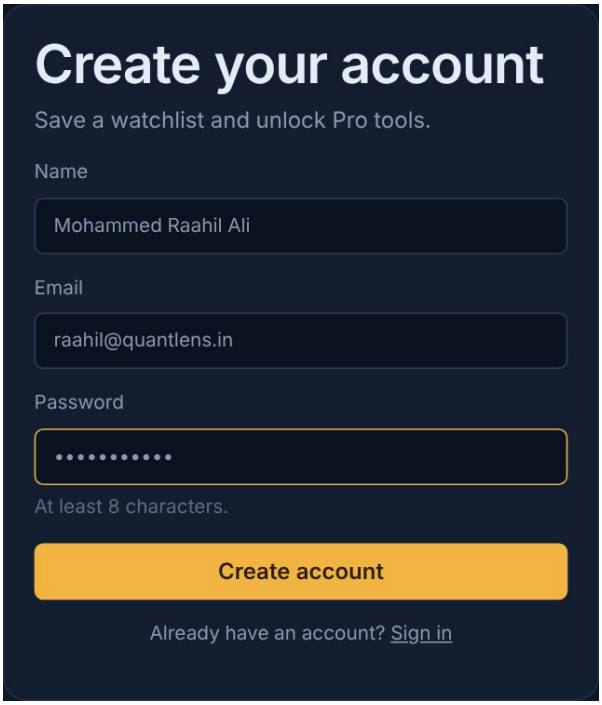


Figure 8: Account creation (JWT)

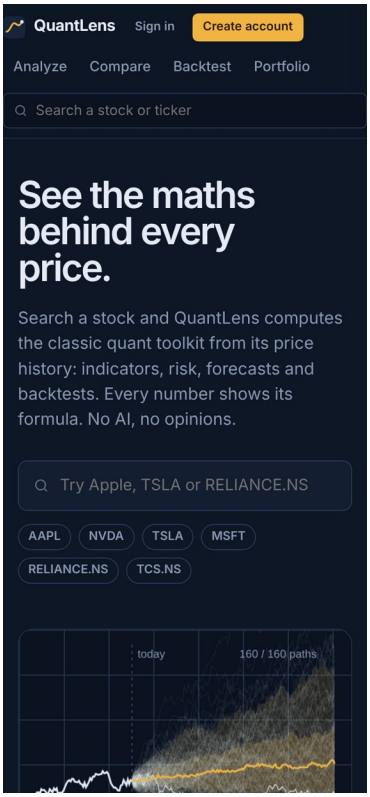


Figure 9: Landing page on a phone

7.2 Stock analysis

Searching a symbol opens its analysis page. The candlestick chart can show any combination of SMA 20/50/200, EMA 12/26, Bollinger Bands and the regression trend, with RSI, MACD or the Stochastic oscillator in the lower pane.



Figure 10: Stock analysis page for AAPL with SMA 50/200, Bollinger Bands and RSI



Figure 11: Same chart with the MACD line, signal line and histogram



Figure 12: Same chart with the Stochastic oscillator and 20/80 levels

Below the chart, the risk panel shows each metric with the formula that produced it, followed by the underwater drawdown chart and plain-language readings of the latest indicator values.



Figure 13: Risk metrics, each shown with its formula



Figure 14: Drawdown chart and latest indicator readings

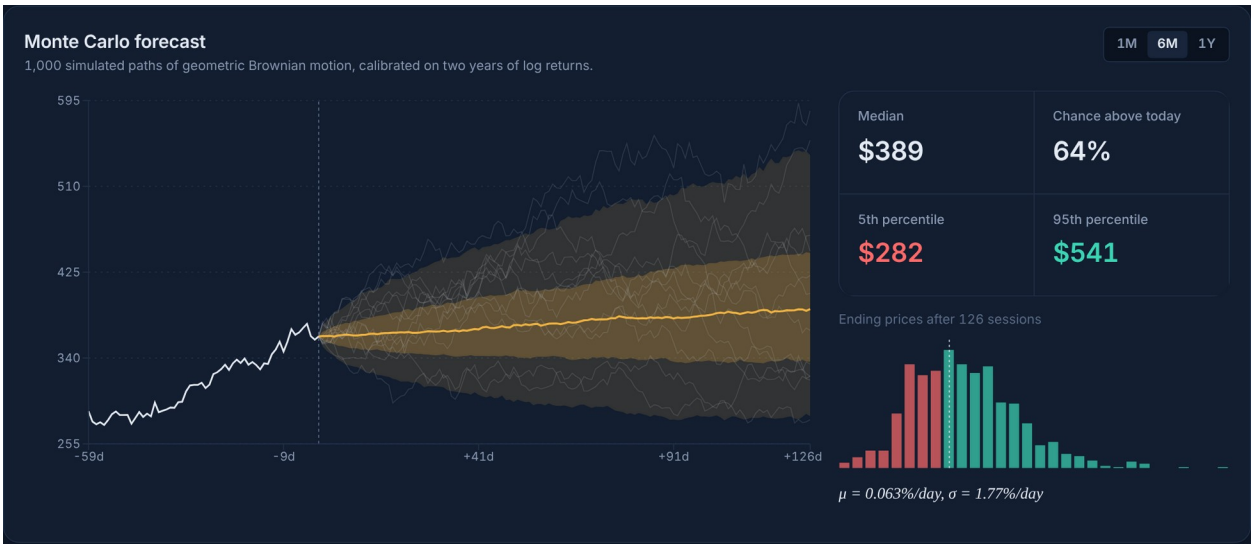


Figure 15: Monte Carlo fan chart (1,000 paths) with the ending-price histogram



Figure 16: Creating a price alert (Pro) from the stock page

7.3 Comparing stocks



Figure 17: Correlation heatmap and growth chart for five stocks

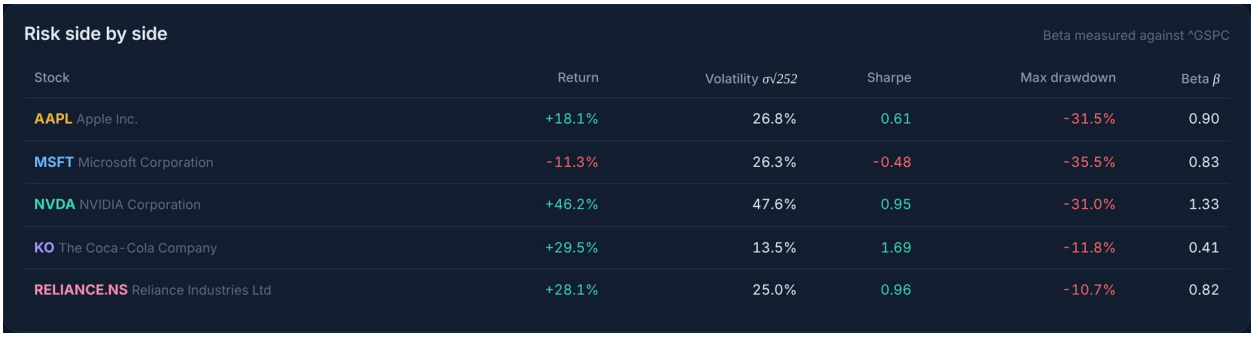


Figure 18: Risk side by side, with beta measured against the S&P 500

7.4 Strategy backtester

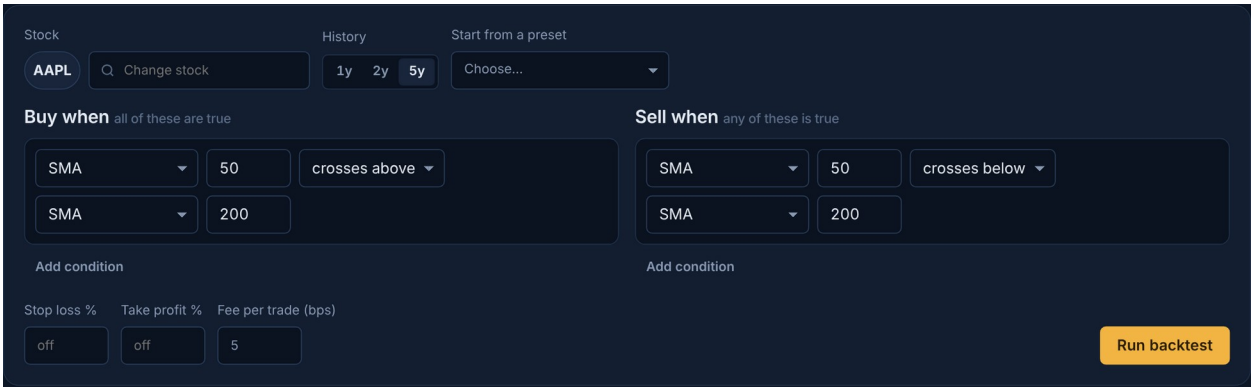


Figure 19: Rule builder: buy when SMA 50 crosses above SMA 200, sell on the reverse cross

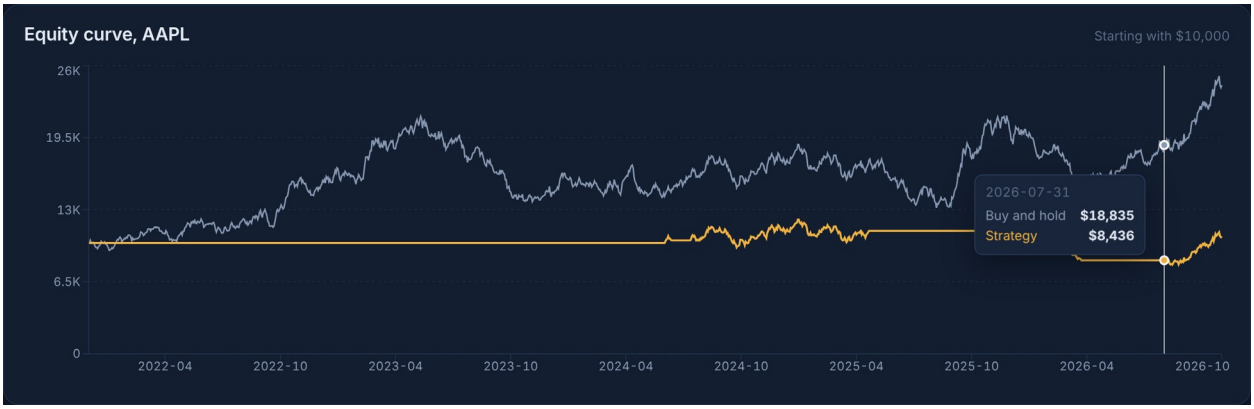


Figure 20: Equity curve of the strategy (amber) against buy-and-hold (grey)

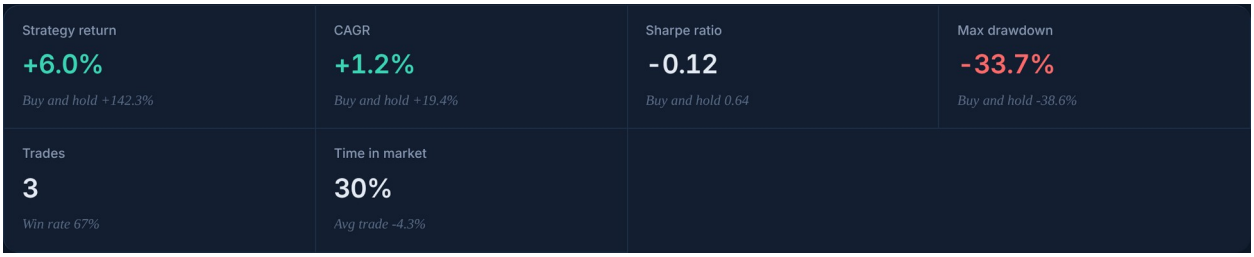


Figure 21: Backtest statistics compared with buy-and-hold

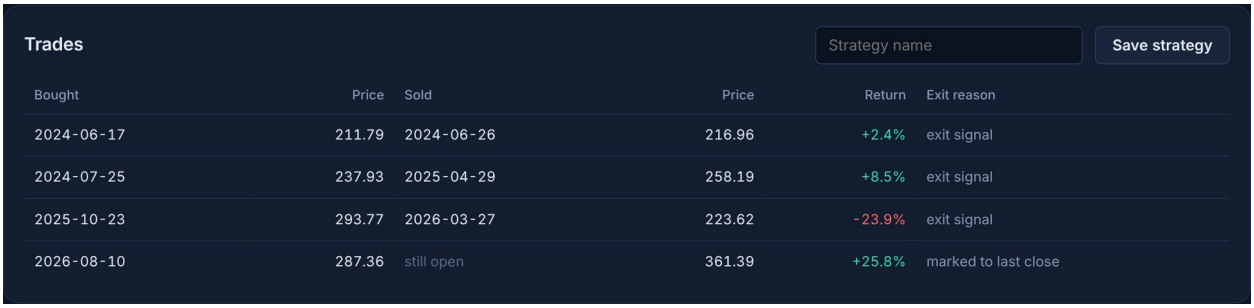


Figure 22: Trade list with entry, exit, return and exit reason

7.5 Portfolio builder

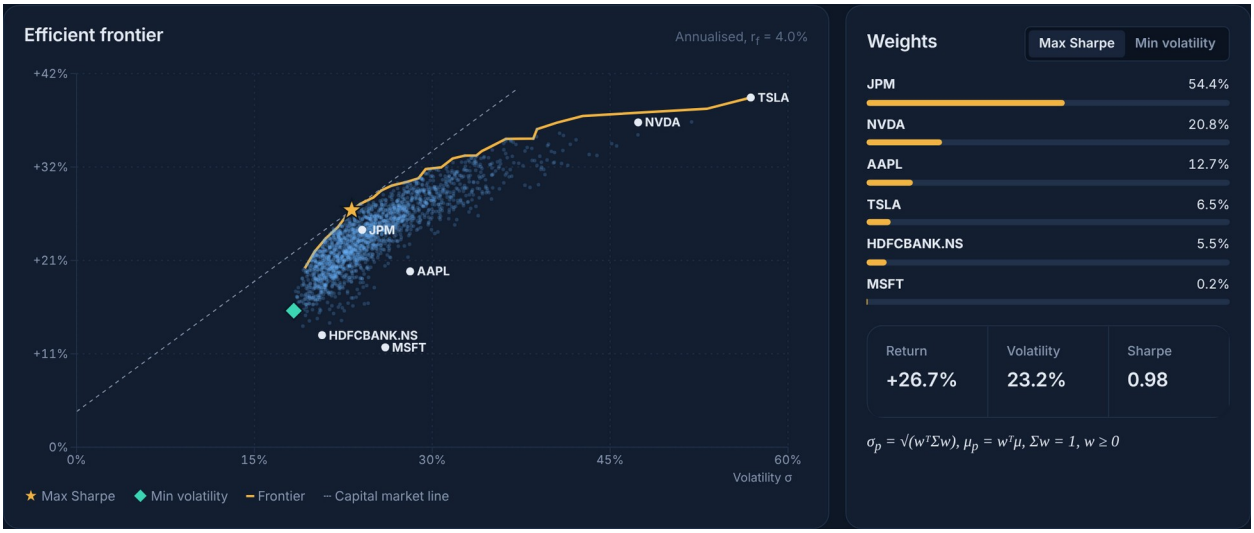


Figure 23: Efficient frontier with max-Sharpe (star) and min-volatility (diamond) portfolios



Figure 24: Correlation matrix of the portfolio's stocks

7.6 Dashboard, Pro plan and mobile layout

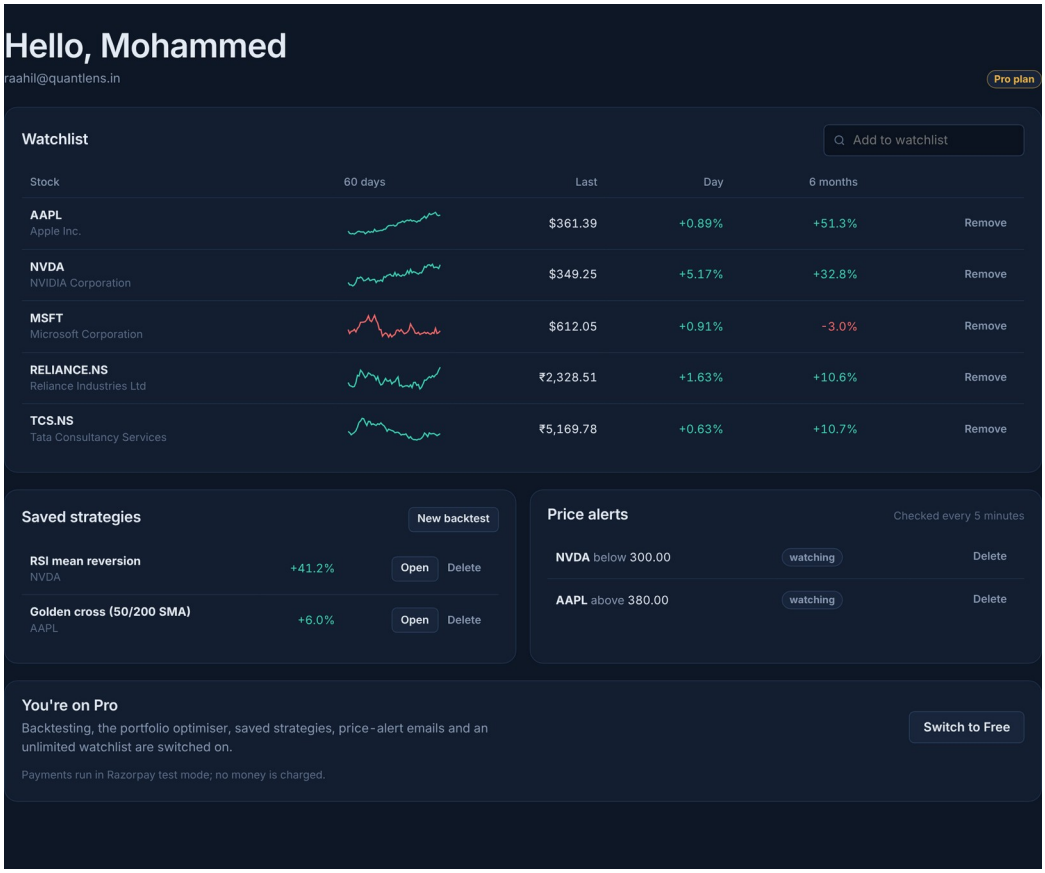


Figure 25: User dashboard: watchlist with sparklines, saved strategies, price alerts and plan

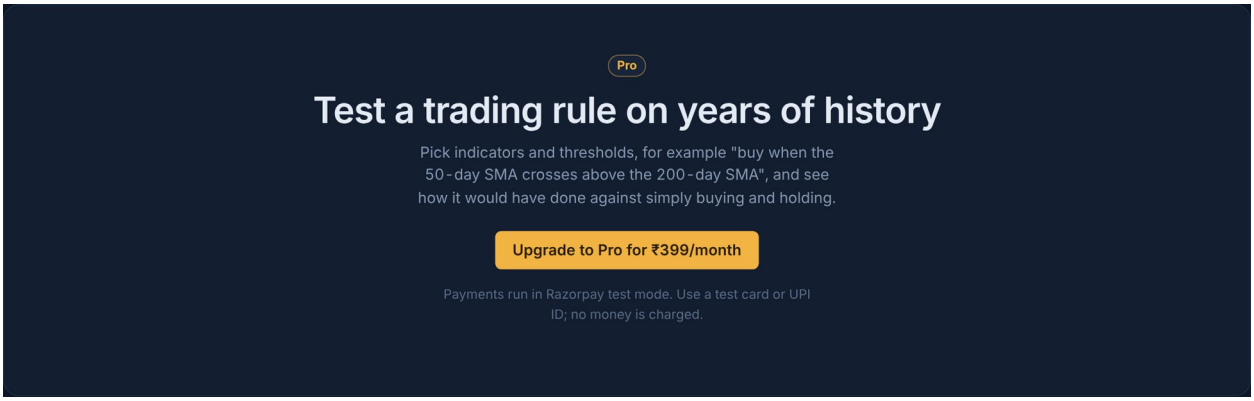


Figure 26: Pro features shown to a Free user, with the Razorpay upgrade button

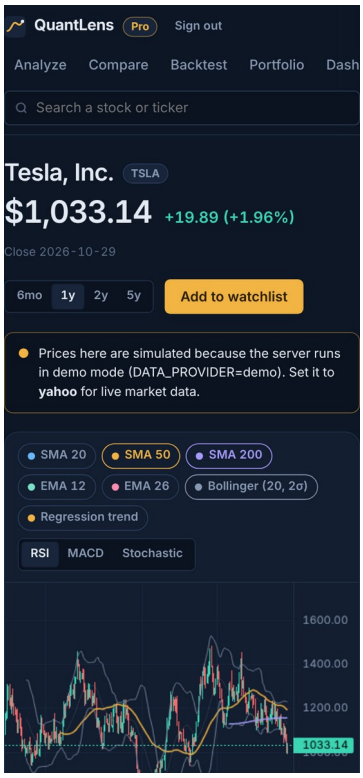


Figure 27: Stock page on a phone

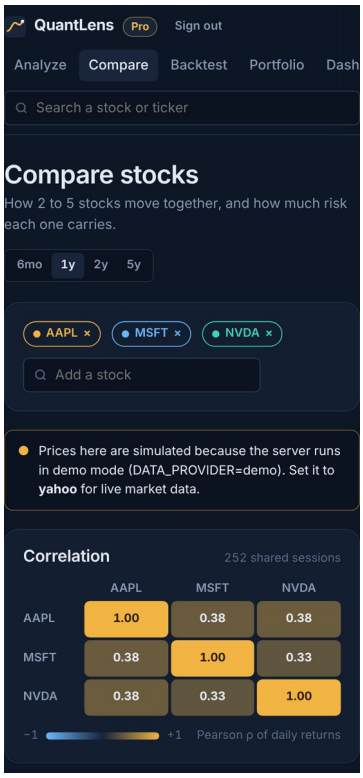


Figure 28: Compare page on a phone

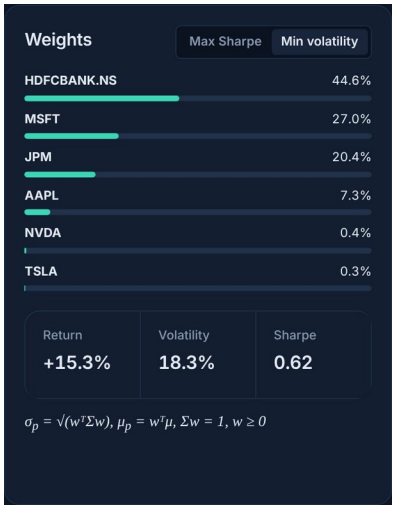


Figure 29: Minimum-volatility weights

8. Results / Findings

8.1 Functional results

Table 10: Feature completion

Feature (in build order)	Status	Verified by
Auth, stock search and price chart	Complete	UI test and API test
Indicator engine (SMA, EMA, RSI, MACD, Bollinger, Stochastic)	Complete	Unit tests against naive reference
Risk metrics and watchlist	Complete	Unit tests, PostgreSQL integration
Strategy backtester	Complete	Unit test (no look-ahead), UI test
Monte Carlo and efficient frontier	Complete	Unit tests, BullMQ integration
Correlation heatmap for 2–5 stocks	Complete	Unit test (symmetric, unit diagonal)
Saved strategies and price-alert emails	Complete	Integration test (alert fired)
Razorpay payment for Pro	Complete	Signature test (forged rejected, valid accepted)
Responsive layout	Complete	Screenshots at 1,280 and 390 pixels

8.2 Automated testing

The math engine has 15 automated tests (npm test in the server folder), and all of them pass. Each optimised algorithm is checked against a brute-force implementation or a value known in advance.

Table 11: Automated test suite

#	Test	What it proves
1	SMA matches the naive window average	Rolling-sum SMA is exact
2	EMA seeds with SMA and follows the recurrence	EMA warm-up and smoothing are correct
3	RSI is 100 on a rising series and within [0, 100]	Wilder smoothing and bounds
4	Bollinger middle = SMA, width = 2σ	Sum-of-squares variance is exact
5	MACD histogram = line − signal	MACD chaining and warm-up length
6	Rolling extrema match brute force	Monotonic deque is correct; %K in range
7	Regression recovers an exact line	Slope, intercept and $R^2 = 1$
8	Max drawdown finds deepest fall	Peak and trough indexes
9	Historical VaR picks the 5th percentile	VaR ≤ expected shortfall
10	Beta of a 2× levered copy is 2	Covariance, variance and volatility
11	Correlation matrix symmetric, unit diagonal	Series and its negative give −1
12	Frontier weights sum to 1	Max-Sharpe beats every sample; frontier rises
13	Monte Carlo bands ordered and reproducible	1,000 paths; same seed, same result
14	Backtest has no look-ahead	Trades fill at the next bar’s open
15	Strategy validation rejects bad input	Unknown indicators and empty rules refused

8.3 Integration testing

The full production stack was run with PostgreSQL, Redis and BullMQ. Registration, login, plan upgrade, watchlist, saved strategies and alerts were written to and read back from PostgreSQL; price series appeared as Redis keys; a backtest ran as a BullMQ job and an identical second request was served instantly from the result cache; and the alert checker detected a crossed target and produced the notification email. For payments, a forged Razorpay signature was rejected and a correctly signed one upgraded the account to Pro.

8.4 Findings

- Rolling-window algorithms make indicator computation linear in the number of trading days, so a five-year series of about 1,260 sessions is analysed in milliseconds.
- Caching is essential with free data APIs: after the first request, every further analysis, comparison or job on the same symbol needs no external call.
- In the example backtest, the 50/200-day golden-cross rule returned +6.0% over five years against +142.3% for buy-and-hold with a similar maximum drawdown, which shows why every strategy must be compared with the passive baseline.

- The efficient frontier shows that combining stocks with low correlation produces portfolios with lower volatility than most individual stocks for the same expected return, which is the central result of Markowitz theory.
- Running heavy computations as background jobs keeps every page responsive, and showing the formula next to each number makes the analysis transparent and suitable for learning.

9. Conclusion

QuantLens meets its objectives as a complete full-stack web application. A responsive React single-page application connects to a secure Express REST API backed by PostgreSQL and Redis, and a tested mathematical engine computes trend, momentum, risk, portfolio and forecasting metrics from price history. Heavy computations run as background jobs, market data is cached to respect API limits, and a Free/Pro model with Razorpay payments shows how the product could be offered commercially. Replacing the random charts and AI verdicts of the earlier version with transparent, reproducible mathematics has made the application more accurate, easier to verify and more useful for learning how markets are measured.

10. Future Enhancement of the Project in Real-Time Applications

Frontend

- Live intraday prices over WebSockets with streaming candle updates on the chart.
- Drag-and-drop strategy builder with more indicators (ATR, ADX, VWAP, Ichimoku) and multi-stock strategies.
- Progressive Web App with an offline watchlist and push notifications for alerts; a light theme and an accessibility audit.
- Downloadable PDF reports of an analysis, a backtest or a portfolio.

Backend

- Razorpay Subscriptions with webhooks for automatic monthly renewal, invoices and cancellation.
- Walk-forward optimisation and parameter sweeps for strategies, run in parallel across worker processes.
- An exact quadratic-programming optimiser for the frontier with constraints such as sector limits and maximum weights.
- Daily prices stored in a time-series database (TimescaleDB) with indicators pre-computed in nightly jobs.
- Deployment with Docker: frontend on Vercel, API and workers on Render or AWS, managed PostgreSQL and Redis, and continuous integration running the test suite on every commit.
- SMS and WhatsApp alerts, and support for mutual funds, ETFs and cryptocurrencies.